

Molecular Dynamics Simulation of Argon Surface Structure and Dynamics Using the Lennard–Jones Potential

Physics 440/679: Computational Methods in Physics with Python

Concordia University

October 25, 2025

Authors:

Asaad Hanna [40240212]

Alessandro Di Marzo [40245309]

Dharshaanan Pushpakaran [40321609]

Erik Hristev [40188837]

1 Introduction

The goal of this experiment is to investigate the structural and dynamical properties of argon at the atomic level using molecular dynamics (MD) simulations based on the Lennard–Jones (LJ) potential. Classical thermodynamic models such as the ideal gas law and the van der Waals equation provide valuable macroscopic insight but fail to account for atomistic details of intermolecular interactions [7, 8]. MD simulations overcome these limitations by explicitly computing particle trajectories governed by realistic force fields, allowing for accurate predictions of thermodynamic and transport properties at the nanoscale [1]. Ultimately, this study aims to validate the MD method for simple fluids by reproducing key bulk and surface properties of LJ argon.

We compute observables including density profiles, radial distribution functions, mean-squared displacements used to extract diffusion coefficients, and energy distributions. For the surface system, additional observables such as surface density profiles and potential energy variations are analyzed.

The LJ potential, first proposed to model van der Waals forces, captures the essential physics of short-range repulsion and long-range attraction, making it especially effective for simple monatomic systems like argon [2]. The essential features of intermolecular forces such as short-range repulsion are useful for modelling overlapping electron clouds as with long-range attractions arising from van der Waals or dispersion forces. The LJ potential energy, $U(r)$, between two particles separated by a distance r , is expressed as

$$U(r) = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right]$$

Here, ϵ and σ set the energy and length scales, respectively, allowing the potential to be expressed in dimensionless (LJ) reduced units that simplify comparison between different systems. The r^{-12} term models the steep repulsive short-range interaction caused by the Pauli exclusion principle, while the r^{-6} term describes the attractive London dispersion forces.

Previous studies have demonstrated that argon’s phase behavior, surface layering,

and diffusion characteristics are well described by LJ-based models when appropriate parameters are employed [3]. Recent refinements of the potential have further improved agreement with experimental data on argon’s liquid–vapor coexistence and surface phenomena [4]. These studies establish the LJ model as a benchmark framework for studying simple fluids and validating computational techniques.

In this experiment, we extend a working MD code to simulate both bulk and surface argon systems using standard LJ simulation parameters. Argon serves as a prototypical simple fluid due to its monatomic nature and well-characterized LJ parameters. The simulation employs periodic boundary conditions in the planar directions and an open boundary along the surface-normal axis to model a realistic liquid–vapor interface. We compute observables including density profiles, radial distribution functions, mean-squared displacements, and energy distributions. By comparing our results with established literature and classical equations of state, we aim to demonstrate how atomistic simulations bridge the gap between microscopic interactions and macroscopic fluid behavior.

2 Computational Tools

For this project, we employ a Python-based MD simulation framework developed and extended from pre-existing course code. In particular, we utilize the `NumPy` [9] and `Matplotlib` [10] libraries for efficient numerical computation and data visualization, respectively. In addition to our Python framework, we employ the `MDTraj` package [6] to load simulation trajectories. The simulation output is written in GROMACS `.gro` file format, allowing us to visualize the MD simulation in software such as Visual Molecular Dynamics (VMD) [5, 11].

These tools allow flexible manipulation of atomic coordinates, forces, and thermodynamic observables while maintaining clear and accessible code for analysis and modification. Key adjustable parameters include:

- number of atoms (N)

- atom mass (m)
- integration time step (Δt)
- cutoff radius (r_{cut})
- system temperature (T)
- lattice size and structure
- velocity resampling frequency in the Andersen thermostat

These allow tuning between bulk and surface simulations and exploration of equilibrium and non-equilibrium regimes. The entire project is housed in a Jupyter notebook running in Python through the Anaconda environment. The notebook runs locally, with final simulation results saved in the working directory as `.gro` files.

3 Algorithm Development

The MD simulation evolves the positions and velocities of N argon atoms in time according to Newton’s equations of motion under the influence of the LJ potential. The implementation builds upon the modular Python Jupyter notebook framework, where each function performs a well-defined role in the integration, force computation, and analysis processes.

3.1 Core Integration Algorithm

Time evolution is performed using the velocity–Verlet integrator, implemented in the function `advance()`. This symplectic algorithm provides second-order accuracy in time and ensures good energy conservation in microcanonical (NVE) ensemble simulations. At each time step, atomic positions and velocities are updated as:

$$\begin{aligned}
\mathbf{v}\left(t + \frac{\Delta t}{2}\right) &= \mathbf{v}(t) + \frac{1}{2}\mathbf{a}(t)\Delta t \\
\mathbf{r}(t + \Delta t) &= \mathbf{r}(t) + \mathbf{v}\left(t + \frac{\Delta t}{2}\right)\Delta t \\
\mathbf{v}(t + \Delta t) &= \mathbf{v}\left(t + \frac{\Delta t}{2}\right) + \frac{1}{2}\mathbf{a}(t + \Delta t)\Delta t
\end{aligned}$$

Periodic boundary conditions are applied through the `minimum_image()` function to maintain continuity across the cubic box, ensuring that particles exiting one side re-enter from the opposite side.

The interatomic forces are computed via the `force()` function, which applies the LJ potential up to a specified cutoff radius $r_c = L/2$. To ensure energy continuity, the potential is shifted to zero at r_c , implemented in the `potential()` function.

3.2 Thermostat Implementation

Temperature control is introduced via the Andersen thermostat, implemented in `thermostat()`. At each step, a small random subset (1%) of particles is selected, and their velocities are reassigned from a Gaussian distribution corresponding to the target temperature T . This mimics stochastic collisions with a heat bath, ensuring sampling of the canonical (NVT) ensemble while preserving realistic dynamics between collisions.

3.3 Initialization and Parameters

Particle coordinates are initialized on a uniform cubic lattice using `cubic_lattice()`, and velocities are assigned via `initial_velocities()` to achieve a desired temperature. The initial velocity distribution is centered (zero net momentum) and rescaled using `get_temperature()` to match the equipartition theorem. To transition this general code to a surface simulation, key adaptations were made to the initialization and parameter choices. The `cubic_lattice()` routine was specifically modified to generate an argon slab geometry with appropriate vacuum spacing. We explored various calculation of non-cubic box dimensions (L_x, L_y, L_z) and the application of semi-periodic boundary conditions (PBCs) only in the x and y planar directions, leaving the z -axis open to

model a free surface. Furthermore, the selection of the reduced temperature is crucial, as it must be controlled precisely via the Andersen thermostat to place the fluid in the liquid-vapor coexistence region, which corresponds to liquid argon near its triple or critical transition temperatures. These simple but strategic changes to the initial conditions and temperature control are essential for defining the system as an argon surface and allowing the spontaneous formation of the liquid-vapor interface.

3.4 Analysis and Post-Processing

Several analysis routines are included to extract physical observables:

- **Radial Distribution Function $g(r)$:** computed by `pair_correlation()` through histogramming all pair separations within the simulation box, normalized by the ideal gas reference density.
- **Structure Factor $S(k)$:** computed from the Fourier transform of the density via `calc_sk()` and averaged over equivalent k-vectors using `calc_av_sk()`.
- **Kinetic and Potential Energies:** evaluated using `kinetic()` and `potential()` at each saved frame to monitor equilibration.
- **Temperature Trajectory:** extracted from instantaneous velocities using `get_temperature()` to verify ensemble stability.

All functions are vectorized using NumPy arrays to maximize computational efficiency and readability.

3.5 Testing and Validation

Code correctness and physical fidelity were verified through multiple diagnostic checks:

- **Energy conservation:** In NVE runs, total energy remains constant with negligible drift, confirming correct integration and force calculations.

- **Thermostat validation:** In NVT runs, the steady-state temperature matches the target temperature, and the velocity distribution reproduces the theoretical Maxwell–Boltzmann form.
- **RDF validation:** The computed $g(r)$ for liquid argon matches benchmark data for the LJ potential.
- **Block averaging:** Implemented in `block_average()` to estimate statistical errors and ensure convergence of time-averaged quantities.

Algorithm Workflow

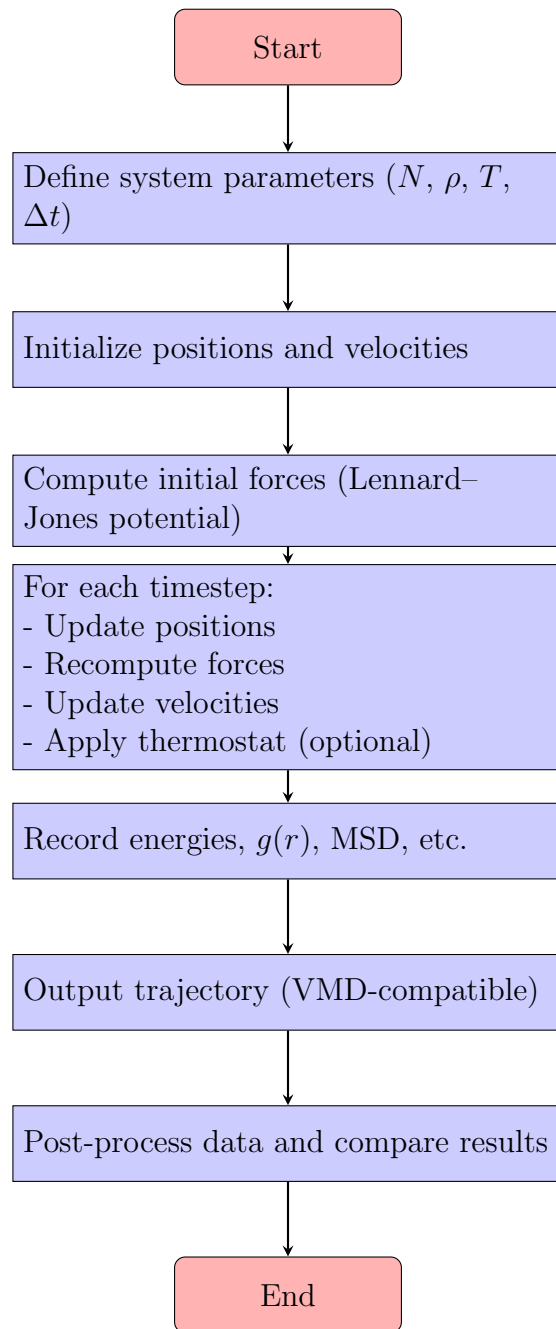


Figure 1: Illustrative workflow of the molecular dynamics algorithm for simulating argon using the Lennard–Jones potential.

4 Procedure

The simulation is conducted entirely in LJ reduced units, where the characteristic length σ , energy ε , and mass m are defined as $\sigma = 1$, $\varepsilon = 1$, and $m = 1$. This non-dimensional convention simplifies parameter tuning, enhances code portability, and ensures results are universally applicable to simple fluids, which can later be rescaled to physical argon properties. The chosen LJ parameters align the simulation’s thermodynamic behavior with the known characteristics of liquid argon.

4.1 Initialization and Equilibration

The procedure begins by defining the system’s size and initial state. We will simulate a system of $N = 640$ atoms, initialized on a face-centered cubic (FCC) lattice truncated along the z -axis to create the liquid slab geometry. The simulation box dimensions are chosen as $L_x = 8\sigma$, $L_y = 8\sigma$, and $L_z = 10\sigma$ to ensure sufficient vacuum spacing above the liquid surface, thereby preventing interactions across the non-periodic z -boundary. Initial velocities are random and later modified by the thermostat which draws from a Maxwell–Boltzmann distribution corresponding to a reduced target temperature of $T = 1.0\varepsilon/k_B$, this is representative of liquid argon near its triple point. A constant integration time step of $\Delta t = 0.005$ reduced units and a cutoff radius of $r_{\text{cut}} = 2.5\sigma$ are employed. The system is first equilibrated over 5,000 steps under the Andersen thermostat to maintain temperature stability and achieve sampling of the canonical ensemble (NVT).

4.2 Production Run and Analysis

Following equilibration, production runs are performed under the same NVT conditions for a total of 50,000 steps. Observables are sampled every 10 steps to ensure statistical independence and compute time-averaged ensemble quantities. The mean-squared displacement (MSD) is calculated to estimate the diffusion constant using the Einstein relation [12]. The density profile $\rho(z)$ is obtained by binning the number of atoms along the z -axis to quantitatively characterize the liquid-vapor interface and surface layering.

Furthermore, we will utilize structural analysis to extract phase-defining properties. The resulting pair-correlation function ($g(r)$) and the structure factor ($S(k)$) are key indicators used to identify the approximate phase transition temperatures, allowing us to compare our results with established literature; for instance, the liquid-gas critical point for an LJ fluid is known to be around $T_c \approx 1.321\epsilon k_B^{-1}$ [13].

4.3 Validation and Conclusion

Simulation correctness of the physical properties is validated through periodic energy conservation checks during the NVE portion of the equilibration phase, and by comparison of bulk quantities, such as the equilibrium density and primary $g(r)$ peaks, with known benchmark data for LJ argon. Once verified, the collected dynamic and structural data will explicitly highlight the influence of the semi-periodic boundaries on the atomic ordering and movement at the liquid-vapor interface. In conclusion, this procedure provides a comprehensive, well-parameterized, and testable framework whose goals are to compute the required structural and dynamic observables of the argon surface and also to serve as validation the fundamental computational physics techniques used to model complex thermodynamic phenomena.

References

- [1] Alder, B. J., & Wainwright, T. E. (1959). *Studies in molecular dynamics. I. General method. The Journal of Chemical Physics*, 31(2), 459–466. <https://doi.org/10.1063/1.1730376>
- [2] Chen, Y., et al. (2022). *A revised Lennard–Jones potential for bubble nucleation study of argon based on the molecular dynamics simulation method. Journal of Molecular Liquids*, 371, 121094. <https://doi.org/10.1016/j.molliq.2022.121094>
- [3] Kulakova, L., et al. (2017). *Data driven inference for the repulsive exponent of the Lennard–Jones potential in molecular dynamics simulations. Scientific Reports*, 7(1). <https://doi.org/10.1038/s41598-017-16314-4>
- [4] Informatics, N. O. O. D. A. (2025). *Argon*. <https://webbook.nist.gov/cgi/cbook.cgi?ID=7440-37-1>
- [5] *What is VMD?* (2008). https://www.ks.uiuc.edu/Research/vmd/allversions/what_is_vmd.html
- [6] McGibbon, R. T., et al. (2015). *MDTraj: A modern open library for the analysis of molecular dynamics trajectories. Biophysical Journal*, 109(8), 1528–1532. <https://doi.org/10.1016/j.bpj.2015.08.015>.
MDTraj — MDTraj 1.9.8.dev0 documentation. (n.d.). <https://www.mdtraj.org/1.9.8.dev0/index.html>
- [7] Verlet, L. (1967). *Computer “Experiments” on Classical Fluids. I. Thermodynamical Properties of Lennard–Jones Molecules. Physical Review*, 159(1), 98–103. <https://doi.org/10.1103/physrev.159.98>
- [8] Frenkel, D., & Smit, B. (2002). *Understanding Molecular Simulation: From Algorithms to Applications*. In *Elsevier eBooks*. <https://doi.org/10.1016/b978-0-12-267351-1.x5000-7>

- [9] NumPy documentation — *NumPy v2.1 Manual*. (2008). <https://numpy.org/doc/2.1/index.html>
- [10] Matplotlib — *Visualization with Python*. (2012). <https://matplotlib.org/>
- [11] VMD — *Visual Molecular Dynamics*. (2025). <https://www.ks.uiuc.edu/Research/vmd/>
- [12] *Mean Square Displacement — GROMACS 2021.1 Documentation*. (2021). <https://manual.gromacs.org/2021.1/reference-manual/analysis/mean-square-displacement.html>
- [13] Stephan, S., Thol, M., Vrabec, J., & Hasse, H. (2019). *Thermophysical properties of the Lennard–Jones fluid: Database and data assessment*. *Journal of Chemical Information and Modeling*, 59(10), 4248–4265. <https://doi.org/10.1021/acs.jcim.9b00620>